

# Unit 5.

## Working with Database

ADO.NET basics: Connection, Command, Reader and Adapter classes, Entity Framework (EF) Core, Object-Relational Mapper (ORM), Adding EF Core to an application: Choosing database provider, data models and data context, Querying and Saving data to database: Create, read, update and delete records

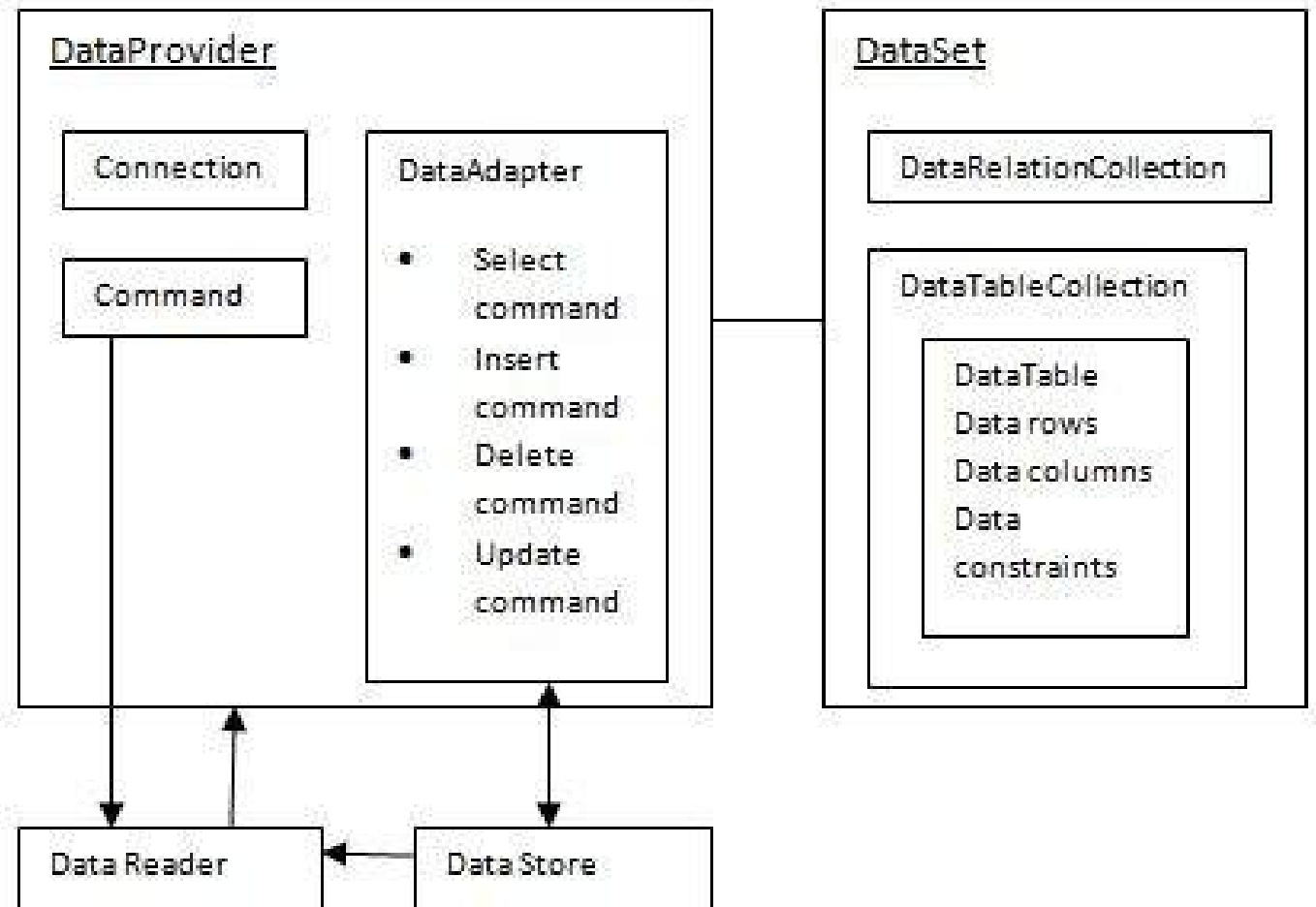
By: Yuba Raj Devkota  
Net Centric Computing  
CSIT 6<sup>th</sup> Semester  
NCCS College

# ADO.NET Basics

**ADO.NET (ActiveX Data Objects .NET)** is a data access technology provided by Microsoft to connect .NET applications with databases such as SQL Server, MySQL, Oracle, and SQLite.

It allows applications to:

- Connect to a database
- Execute SQL queries
- Read records
- Insert, Update and Delete data
- Work with disconnected data



*ADO.NET Architecture*

# Main Components of ADO.NET

## 1. Connection Class

1. Connection
2. Command
3. DataReader
4. DataAdapter
5. DataSet (optional)

A **Connection** object establishes communication between the application and the database.

Without a connection, no database operation is possible.

For SQL Server, the class used is

`SqlConnection`

Namespace

`</> C#`

```
using Microsoft.Data.SqlClient;
```

### Connection String

A connection string contains information required to connect to a database.

Example

```
Server=localhost;  
Database=CollegeDB;  
Trusted_Connection=True;  
TrustServerCertificate=True;
```

```
using Microsoft.Data.SqlClient;

string conString =
"Server=localhost;Database=CollegeDB;Trusted_Connection=True;TrustServerCertificate=True";

SqlConnection con = new SqlConnection(conString);

con.Open();

Console.WriteLine("Database Connected");

con.Close();
```

## Output

Database Connected

## Important Methods

| Method    | Description               |
|-----------|---------------------------|
| Open()    | Opens database connection |
| Close()   | Closes connection         |
| Dispose() | Releases resources        |

## 2. Command Class

Example: Insert Data

*The Command object executes SQL statements against the database.*

Class used

SqlCommand

### Types of Commands

- SELECT
- INSERT
- UPDATE
- DELETE
- Stored Procedure

```
using Microsoft.Data.SqlClient;

string conString =
"Server=localhost;Database=CollegeDB;Trusted_Connection=True;TrustServerCertificate=True";

SqlConnection con = new SqlConnection(conString);

con.Open();

string sql =
"INSERT INTO Student(Name, Age) VALUES('Ram', 20)";

SqlCommand cmd = new SqlCommand(sql, con);

cmd.ExecuteNonQuery();

Console.WriteLine("Record Inserted");

con.Close();
```

## Execute Methods

| Method            | Used For               |
|-------------------|------------------------|
| ExecuteNonQuery() | INSERT, UPDATE, DELETE |
| ExecuteScalar()   | Returns single value   |
| ExecuteReader()   | Returns multiple rows  |

## Example of ExecuteScalar()

Suppose we want total students.

```
</> C#
```

```
string sql =  
"SELECT COUNT(*) FROM Student";  
  
SqlCommand cmd = new SqlCommand(sql, con);  
  
int total = (int)cmd.ExecuteScalar();  
  
Console.WriteLine(total);
```

Output

35

### 3. DataReader Class

A **DataReader** reads records one by one from the database.

Class

SqlDataReader

Working

Output

It is

- Fast
- Forward-only
- Read-only

Database

|  
SqlCommand

|  
ExecuteReader()

|  
SqlDataReader

|  
Read() → Read() → Read()

Ram 20  
Hari 22  
Sita 19

```
SqlConnection con =  
new SqlConnection(connectionString);  
  
con.Open();  
  
string sql =  
"SELECT * FROM Student";  
  
SqlCommand cmd =  
new SqlCommand(sql, con);  
  
SqlDataReader reader =  
cmd.ExecuteReader();  
  
while(reader.Read())  
{  
    Console.WriteLine(  
        reader["Name"] + " " +  
        reader["Age"]);  
}  
  
reader.Close();  
  
con.Close();
```

#### Advantages

- Very fast
- Uses less memory
- Good for displaying data

#### Disadvantages

- Read-only
- Cannot move backward
- Requires active connection

## 4. DataAdapter Class

A **DataAdapter** transfers data between a database and a **DataSet**.

Unlike **DataReader**, it works in **Disconnected Mode**.

Class

SqlDataAdapter

### Working

```
graph TD; Database --> SqlDataAdapter; SqlDataAdapter --> Fill; Fill --> DataSet; DataSet --> Application;
```

Database  
|  
SqlDataAdapter  
|  
Fill()  
|  
DataSet  
|  
Application

### Advantages

- Disconnected architecture
- Can edit data
- Can work offline

```
SqlConnection con =  
new SqlConnection(connectionString);  
  
string sql =  
"SELECT * FROM Student";  
  
SqlDataAdapter da =  
new SqlDataAdapter(sql, con);  
  
DataSet ds =  
new DataSet();  
  
da.Fill(ds);  
  
Console.WriteLine(  
ds.Tables[0].Rows.Count);
```

Output

25



*Difference: DataReader vs DataAdapter*

| DataReader   | DataAdapter     |
|--------------|-----------------|
| Connected    | Disconnected    |
| Fast         | Slightly slower |
| Read-only    | Read and Write  |
| Forward only | Random access   |
| Low memory   | More memory     |

*Summary of ADO.NET Classes*

| Class          | Purpose                  |
|----------------|--------------------------|
| SqlConnection  | Connect database         |
| SqlCommand     | Execute SQL              |
| SqlDataReader  | Read data                |
| SqlDataAdapter | Transfer data to DataSet |
| DataSet        | Store disconnected data  |

## Example: ADO.NET to connect, insert and Retrieve data using mySQL

← ↻ <https://localhost:7180/Student>

---

AdoNetDemo [Home](#) [Privacy](#)

---

# Student List

[Add Student](#)

| Id | Name | Email         | Age |
|----|------|---------------|-----|
| 0  | Ram  | ram@gmail.com | 12  |

---

AdoNetDemo [Home](#) [Privacy](#)

---

# Add Student

Name:

Email:

Age:

[View Students](#)

Browse

Structure

SQL

Search

Insert

✓ Table students has been altered successfully.

```
ALTER TABLE `students` CHANGE `age` `Age` INT(11) NOT NULL;
```

[Edit inline](#) [\[ Edit \]](#) [\[ Create PHP code \]](#)
 Table structure

 Relation view

| #                          | Name                                                                                        | Type         | Collation          | Attributes | Null | Default | C |
|----------------------------|---------------------------------------------------------------------------------------------|--------------|--------------------|------------|------|---------|---|
| <input type="checkbox"/> 1 | <b>Id</b>  | int(11)      |                    |            | No   | None    |   |
| <input type="checkbox"/> 2 | <b>Name</b>                                                                                 | varchar(100) | utf8mb4_general_ci |            | No   | None    |   |
| <input type="checkbox"/> 3 | <b>Email</b>                                                                                | varchar(100) | utf8mb4_general_ci |            | No   | None    |   |
| <input type="checkbox"/> 4 | <b>Age</b>                                                                                  | int(11)      |                    |            | No   | None    |   |

## Step 1: Start MySQL in XAMPP

Open **XAMPP Control Panel** and start:

- Apache
- MySQL

Then open:

```
http://localhost/phpmyadmin
```

## Step 2: Create Database

In phpMyAdmin, click **SQL** and execute:

&lt;/&gt; SQL

```
CREATE DATABASE studentdb;
```

```
USE studentdb;
```

```
CREATE TABLE students (
    Id INT AUTO_INCREMENT PRIMARY KEY,
    Name VARCHAR(100),
    Email VARCHAR(100),
    Age INT
);
```

Your database structure is:

```
studentdb
├── students
│   ├── Id
│   ├── Name
│   ├── Email
│   └── Age
```

## Step 3: Create ASP.NET Core MVC Project

In Visual Studio:

Create a new project → ASP.NET Core Web App (Model-View-Controller)

For example:

Project Name: AdoNetDemo

Choose your .NET version and create the project.

## Step 4: Install MySQL ADO.NET Package

Right-click your project:

Manage NuGet Packages → Browse

Search:

MySQL.Data

Install **MySQL.Data**.

This provides the ADO.NET classes needed to connect ASP.NET Core to MySQL.

## Step 5: Create Student Model

Create:

Models/Student.cs

Put:

```
</> C#  
  
namespace AdoNetDemo.Models  
{  
    public class Student  
    {  
        public int Id { get; set; }  
  
        public string Name { get; set; }  
  
        public string Email { get; set; }  
  
        public int Age { get; set; }  
    }  
}
```

## Step 6: Create Student Controller

Go to:

Controllers

Add:

StudentController.cs

Put the following code:

</> C#

```
using Microsoft.AspNetCore.Mvc;
using MySql.Data.MySqlClient;
using AdoNetDemo.Models;

namespace AdoNetDemo.Controllers
{
    public class StudentController : Controller
    {
        // Database connection
        string connectionString =
            "Server=localhost;Database=studentdb;Uid=root;Pwd=";
```

```
// Display all students
public IActionResult Index()
{
    List<Student> students = new List<Student>();

    using (MySqlConnection con = new MySqlConnection(connectionString))
    {
        con.Open();

        string query = "SELECT * FROM students";

        MySqlCommand cmd = new MySqlCommand(query, con);

        MySqlDataReader reader = cmd.ExecuteReader();

        while (reader.Read())
        {
            Student student = new Student();

            student.Id = Convert.ToInt32(reader["Id"]);
            student.Name = reader["Name"].ToString();
            student.Email = reader["Email"].ToString();
            student.Age = Convert.ToInt32(reader["Age"]);

            students.Add(student);
        }
    }
}
```



```
        return View(students);
    }

    // Show insert form
    public IActionResult Create()
    {
        return View();
    }

    // Insert data
    [HttpPost]
    public IActionResult Create(Student student)
    {
        using (MySqlConnection con = new MySqlConnection(connectionString))
        {
            con.Open();

            string query =
                "INSERT INTO students (Name, Email, Age) VALUES (@Name, @Email, @Age)";

            MySqlCommand cmd = new MySqlCommand(query, con);

            cmd.Parameters.AddWithValue("@Name", student.Name);
            cmd.Parameters.AddWithValue("@Email", student.Email);
            cmd.Parameters.AddWithValue("@Age", student.Age);
```

```
            cmd.ExecuteNonQuery();
        }

        return RedirectToAction("Index");
    }
}
```

## Step 7: Create the Index View

Create this folder if it doesn't already exist:

Views/Student

Inside it create:

Index.cshtml

Put:

HTML

```
<br />
<br />

<table border="1" cellpadding="10">
  <tr>
    <th>Id</th>
    <th>Name</th>
    <th>Email</th>
    <th>Age</th>
  </tr>
```

```
@foreach (var student in Model)
{
  <tr>
    <td>@student.Id</td>
    <td>@student.Name</td>
    <td>@student.Email</td>
    <td>@student.Age</td>
  </tr>
}
</table>
```

## Step 8: Create the Create View

In the same folder:

Views/Student/Create.cshtml

Put:

HTML

```
<form method="post">

  <label>Name:</label>
  <input type="text" name="Name" />
  <br /><br />

  <label>Email:</label>
  <input type="email" name="Email" />
  <br /><br />

  <label>Age:</label>
  <input type="number" name="Age" />
  <br /><br />

  <button type="submit">Save</button>

</form>

<br />

<a href="/Student/Index">View Students</a>
```

## Step 9: Run the Application

Run your ASP.NET Core application.

Suppose your application URL is:

https://localhost:7019

Open:

https://localhost:7019/Student

You should see:

Student List

Add Student

-----

Id      Name              Email              Age

-----

Click:

Add Student

←

↺

https://localhost:7180/Student

AdoNetDemo   Home   Privacy

## Student List

[Add Student](#)

| Id | Name | Email         | Age |
|----|------|---------------|-----|
| 0  | Ram  | ram@gmail.com | 12  |

AdoNetDemo   Home   Privacy

## Add Student

Name:

Email:

Age:

Save

[View Students](#)



# Entity Framework (EF) Core

- *Entity Framework (EF) Core is Microsoft's modern Object-Relational Mapper (ORM) for .NET.*
- *Instead of writing SQL manually, developers work with C# objects (entities), and EF Core translates those operations into SQL queries automatically.*

## Why EF Core?

Without EF Core:

```
</> C#  
  
string sql =  
"SELECT * FROM Student";
```

You must write SQL manually.

With EF Core:

```
</> C#  
  
var students = context.Students.ToList();
```

EF Core generates the SQL behind the scenes.

## EF Core Architecture

```
C# Objects  
|  
Entity Framework Core  
|  
Generated SQL  
|  
SQL Server Database
```

## Important Components

| Component | Description                            |
|-----------|----------------------------------------|
| Entity    | C# class representing a table          |
| DbContext | Session with the database              |
| DbSet     | Represents a table                     |
| Migration | Creates or updates the database schema |

# Example Entity

Student.cs

</> C#

```
public class Student
{
    public int Id { get; set; }

    public string Name { get; set; }

    public int Age { get; set; }
}
```

This class maps to the **Student** table.

# DbContext Example

</> C#

```
using Microsoft.EntityFrameworkCore;

public class CollegeContext : DbContext
{
    public DbSet<Student> Students
    {
        get;
        set;
    }

    protected override void OnConfiguring(
        DbContextOptionsBuilder options)
    {
        options.UseSqlServer(
            "Server=localhost;Database=CollegeDB;Trusted_Connection=True;TrustServerCertificate=True");
    }
}
```

## Insert Record

```
</> C#  
  
CollegeContext db =  
new CollegeContext();  
  
Student s =  
new Student();  
  
s.Name = "Ram";  
  
s.Age = 20;  
  
db.Students.Add(s);  
  
db.SaveChanges();
```

### Generated SQL

```
</> SQL  
  
INSERT INTO Student(Name, Age)  
VALUES('Ram', 20)
```

## Read Records

```
</> C#  
  
CollegeContext db =  
new CollegeContext();  
  
var students =  
db.Students.ToList();  
  
foreach(var s in students)  
{  
    Console.WriteLine(  
        s.Name);  
}
```

### Output

```
Ram  
Hari  
Sita
```

## Update Record

```
</> C#  
  
var student =  
db.Students.Find(1);  
  
student.Age = 25;  
  
db.SaveChanges();
```

### Generated SQL

```
</> SQL  
  
UPDATE Student  
SET Age=25  
WHERE Id=1
```

## Delete Record

`</> C#`

```
var student =  
db.Students.Find(1);  
  
db.Students.Remove(student);  
  
db.SaveChanges();
```

### Generated SQL

`</> SQL`

```
DELETE FROM Student  
WHERE Id=1
```

## LINQ Query Example

Students older than 20.

`</> C#`

```
var result =  
db.Students  
.Where(s => s.Age > 20)  
.ToList();
```

### Equivalent SQL

`</> SQL`

```
SELECT *  
FROM Student  
WHERE Age > 20
```

# Adding EF Core to an Application

- *Entity Framework (EF) Core is a lightweight, open-source Object-Relational Mapper (ORM) that allows developers to work with databases using C# objects instead of writing SQL queries manually.*
- *Adding EF Core to an ASP.NET Core application mainly involves three steps:*

## 1. Create the database in XAMPP

Start **Apache** and **MySQL** in XAMPP.

Open phpMyAdmin and create a database:

SQL

```
CREATE DATABASE college_db;
```

You don't need to create the `students` table manually. **EF Core will create it for us.**

## 2. Install EF Core packages

In Visual Studio:

Tools → NuGet Package Manager → Package Manager Console

Run:

</> PowerShell

```
Install-Package Microsoft.EntityFrameworkCore
Install-Package Microsoft.EntityFrameworkCore.Tools
Install-Package Pomelo.EntityFrameworkCore.MySql
```

The important package for MySQL is:

Pomelo.EntityFrameworkCore.MySql

EF Core will create approximately:

Students

|       |         |             |
|-------|---------|-------------|
| ----- |         |             |
| Id    | INT     | Primary Key |
| Name  | VARCHAR |             |
| Email | VARCHAR |             |
| Age   | INT     |             |

## 3. Create the Model

Create:

Models/Student.cs

```
</> C#

namespace EFCoreDemo.Models
{
    public class Student
    {
        public int Id { get; set; }

        public string Name { get; set; }

        public string Email { get; set; }

        public int Age { get; set; }
    }
}
```

This class represents our database table.



## 4. Create the DbContext

Create a folder called **Data**.

Inside it create:

**Data/ApplicationDbContext.cs**

```
</> C#  
  
using Microsoft.EntityFrameworkCore;  
using EFCoreDemo.Models;  
  
namespace EFCoreDemo.Data  
{  
    public class ApplicationDbContext : DbContext  
    {  
        public ApplicationDbContext(  
            DbContextOptions<ApplicationDbContext> options)  
            : base(options)  
        {  
        }  
  
        public DbSet<Student> Students { get; set; }  
    }  
}
```

## What is DbContext?

*DbContext is the main class that connects your C# application with the database.*

This line:

```
</> C#  
  
public DbSet<Student> Students { get; set; }
```

tells EF Core:

*"I have a Student model and I want an Students table in the database."*



## 5. Add Connection String

Open:

**appsettings.json**

Add:

</> JSON

```
{
  "ConnectionStrings": {
    "DefaultConnection": "server=localhost;database=college_db;user=root;password=;"
  },

  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },

  "AllowedHosts": "*"
}
```

For the normal XAMPP MySQL installation, the username is usually:

root

and the password is usually empty.

So:

user=root;password=;

If you have configured a MySQL password, put it after `password=`.

## 6. Configure EF Core in Program.cs

Open:

Program.cs

Add:

</> C#

```
using Microsoft.EntityFrameworkCore;  
using EFCoreDemo.Data;
```

```
var builder = WebApplication.CreateBuilder(args);
```

```
// Add EF Core
```

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>  
    options.UseMySQL(  
        builder.Configuration.GetConnectionString("DefaultConnection"),  
        ServerVersion.AutoDetect(  
            builder.Configuration.GetConnectionString("DefaultConnection")  
        )  
    ));
```

```
builder.Services.AddControllersWithViews();
```

```
var app = builder.Build();
```

```
app.UseHttpsRedirection();  
app.UseStaticFiles();
```

```
app.UseRouting();
```

```
app.MapControllerRoute(  
    name: "default",  
    pattern: "{controller=Student}/{action=Index}/{id?}");
```

```
app.Run();
```

Now your application knows how to connect to MySQL through EF Core.

## 7. Create the Database Table using EF Core

Open:

Tools → NuGet Package Manager → Package Manager Console

Run:

```
</> PowerShell
```

```
Add-Migration InitialCreate
```

Then:

```
</> PowerShell
```

```
Update-Database
```

EF Core will now create the `Students` table inside:

```
college_db
```

You can check it in:

phpMyAdmin → college\_db → Students

```

using Microsoft.AspNetCore.Mvc;
using EFCoreDemo.Data;
using EFCoreDemo.Models;

namespace EFCoreDemo.Controllers
{
    public class StudentController : Controller
    {
        private readonly ApplicationDbContext _context;

        public StudentController(ApplicationDbContext context)
        {
            _context = context;
        }

        // Display students
        public IActionResult Index()
        {
            var students = _context.Students.ToList();

            return View(students);
        }

        // Show insert form
        public IActionResult Create()
        {
            return View();
        }
    }
}

```

## 8. Create Student Controller

Create:

Controllers/StudentController.cs

```

        // Insert student
        [HttpPost]
        public IActionResult Create(Student student)
        {
            _context.Students.Add(student);

            _context.SaveChanges();

            return RedirectToAction("Index");
        }
    }
}

```

The important EF Core commands are:

**Insert**

</> C#

```

_context.Students.Add(student);
_context.SaveChanges();

```

**Retrieve**

</> C#

```

var students = _context.Students.ToList();

```

```
@model EFCoreDemo.Models.Student
```

```
<h2>Add Student</h2>
```

```
<form asp-action="Create" method="post">
```

```
    <div>
```

```
        <label>Name</label>
```

```
        <input asp-for="Name" />
```

```
    </div>
```

```
    <br />
```

```
    <div>
```

```
        <label>Email</label>
```

```
        <input asp-for="Email" />
```

```
    </div>
```

```
    <br />
```

## 9. Create the Insert View

Create folder:

Views

└─ Student

Inside it create:

Create.cshtml

```
    <div>
```

```
        <label>Age</label>
```

```
        <input asp-for="Age" />
```

```
    </div>
```

```
    <br />
```

```
    <button type="submit">Save Student</button>
```

```
</form>
```

```
<br />
```

```
<a asp-action="Index">View Students</a>
```



```
@model IEnumerable<EFCoreDemo.Models.Student>
```

```
<h2>Student List</h2>
```

```
<a asp-action="Create">Add New Student</a>
```

```
<br />
```

```
<br />
```

```
<table border="1" cellpadding="10">
```

```
<tr>
```

```
<th>Id</th>
```

```
<th>Name</th>
```

```
<th>Email</th>
```

```
<th>Age</th>
```

```
</tr>
```

```
@foreach (var student in Model)
```

```
{
```

```
<tr>
```

```
<td>@student.Id</td>
```

```
<td>@student.Name</td>
```

```
<td>@student.Email</td>
```

```
<td>@student.Age</td>
```

```
</tr>
```

```
}
```

```
</table>
```

## 10. Create the Retrieve/View Page

Create:

Views/Student/Index.cshtml

## 11. Run the Application

Run the project.

Go to:

<https://localhost:xxxx/Student>

You will see:

Student List

Add New Student

| Id | Name | Email | Age |
|----|------|-------|-----|
|----|------|-------|-----|

-----

Click:

Add New Student

# Object-Relational Mapper (ORM)

*ORM (Object-Relational Mapper) is a software technique that allows a programming language to work with a relational database using objects/classes instead of writing SQL queries manually.*

**ORM converts database tables into programming objects and objects into database records.**

## Main Tasks of ORM

### 1. Maps classes to database tables

- `Student` class → `Students` table

### 2. Maps class properties to table columns

- `Student.Name` → `Name` column

### 3. Performs CRUD operations

- Create → Insert
- Read → Select
- Update → Update
- Delete → Delete

### 4. Generates SQL queries automatically

- The programmer can work with objects rather than writing SQL for every operation.



## Example: Entity Framework Core

*ORM = A bridge between object-oriented programming and relational databases.*

Suppose we have a database table:

| Id | Name | Age |
|----|------|-----|
| 1  | Ram  | 20  |
| 2  | Sita | 21  |

In C#, we create a class:

```
public class Student
{
    public int Id { get; set; }
    public string Name { get; set; }
    public int Age { get; set; }
}
```

*Using Entity Framework Core (EF Core), this class can be mapped to the student table*

Instead of writing:

```
</> SQL
```

```
SELECT * FROM Students;
```

we can write:

```
</> C#
```

```
var students = context.Students.ToList();
```

*EF Core (the ORM) translates this into an appropriate SQL query and retrieves the records.*

**ORM is used mostly in Entity Framework Core in .NET, Hibernate in Java, and Django ORM in Python.**

*Create a table named BAG(ID, Brand, Price, Date). Perform the following operations using Entity Framework Core.*

- a. Update the Price of those Book to 2000 which are published on 2022.*
- b. Delete the Book which are published in 1981*
- c. Retrieve the Brand of Book having price greater than 500*

Assume the model is:

**BAG(ID, Brand, Price, Date)**

</> C#

```
using Microsoft.EntityFrameworkCore;

class BAG
{
    public int ID { get; set; }
    public string Brand { get; set; }
    public int Price { get; set; }
    public int Date { get; set; }
}
```

```
class BagContext : DbContext
{
    public DbSet<BAG> Bags { get; set; }

    protected override void OnConfiguring(DbContextOptionsBuilder options)
        => options.UseMySQL(
            "server=localhost;database=testdb;user=root;password=",
            ServerVersion.AutoDetect("server=localhost;database=testdb;user=root;password="));
}
```

```
class Program
{
    static void Main()
    {
        using var db = new BagContext();

        // a. Update Price to 2000 for books published in 2022
        var books = db.Bags.Where(b => b.Date == 2022).ToList();

        foreach (var b in books)
            b.Price = 2000;

        db.SaveChanges();
    }
}
```

```
// b. Delete books published in 1981
var oldBooks = db.Bags.Where(b => b.Date == 1981).ToList();

db.Bags.RemoveRange(oldBooks);
db.SaveChanges();

// c. Retrieve Brand where Price > 500
var result = db.Bags
    .Where(b => b.Price > 500)
    .Select(b => b.Brand)
    .ToList();

foreach (var brand in result)
    Console.WriteLine(brand);
}
}
```

b) What is ADO .Net? Explain the use of Connection, Command and DataReader objects in connecting to a database and retrieving results with an example.

***Assume a database named "IOST" with a table FACULTY (Course\_Name, CourseID, No\_of\_Semester, Fee). Insert some records using ADO.NET and retrieve the fee of records having name CSIT.***

1. How do you cache the information of the client and what are its uses? Create a table named BAG(ID, Brand, Price, Date). Perform the following operations using Entity Framework Core. [3 + 7]

***What is the task of ORM?***

- a. Update the Price of those Book to 2000 which are published on 2022.
- b. Delete the Book which are published in 1981
- c. Retrieve the Brand of Book having price greater than 500

***What is the role of adapter class in ADO.NET? List some features of URL routing.***

3. What are the advantages of stateless HTTP in ASP.NET core application? Create a method to insert a record (3, "John , 12000") onto the table Employee having fields EmployeeId(int), Name varchar(200), Salary(int) using Entity Framework.[4+6]

2. Explain the architecture and design principles of .NET. Create methods to insert, update, delete and read all data for the table Student having following fields StudentId(int), Name varchar(200), RollNo (int), Class varchar(50) using Entity Framework.

10. Express the format of request and response message format. What is the role of adapter class in database connection?